

WEEK 6:PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

-ABINAYA R (E25AI102)

OBJECTIVE

To design and implement a Product Review and Rating Management System that allows customers to submit reviews and ratings for purchased products, securely stores and manages feedback, and uses SQL queries and aggregate functions to analyze product ratings, identify highly rated products, and generate useful customer feedback reports.

1.Create Review and Rating Tables:

```
SQL> CREATE TABLE REVIEW(ReviewID VARCHAR2(7),CustomerID VARCHAR2(7),ProductID VARCHAR2(7),Rating NUMBER(1),ReviewText VARCHAR2(50),ReviewDate DATE,PRIMARY KEY(ReviewID),FOREIGN KEY(CustomerID) REFERENCES CUSTOMER(Customer_ID),FOREIGN KEY(ProductID) REFERENCES PRODUCT(Product_ID));
```

Table created.

```
SQL> DESC REVIEW;
```

Name	Null?	Type
REVIEWID	NOT NULL	VARCHAR2(7)
CUSTOMERID		VARCHAR2(7)
PRODUCTID		VARCHAR2(7)
RATING		NUMBER(1)
REVIEWTEXT		VARCHAR2(50)
REVIEWDATE		DATE

Store Customer Feedback and Ratings

2.Add reviews for purchased products.-Inserting values into the table

```
SQL> INSERT INTO REVIEW VALUES ('R00001','C001','P101',5,'Excellent medicine quality','10-JAN-2026');
```

1 row created.

```
SQL> INSERT INTO REVIEW VALUES ('R00002','C002','P102',4,'Good product and effective','12-JAN-2026');
```

1 row created.

```
SQL> INSERT INTO REVIEW VALUES ('R00003','C003','P103',5,'Very useful medicine','15-JAN-2026');
```

1 row created.

```
SQL> INSERT INTO REVIEW VALUES ('R00004','C004','P104',4,'Good quality product','18-JAN-2026');
```

1 row created.

```
SQL> INSERT INTO REVIEW VALUES ('R00005','C005','P105',5,'Highly satisfied with product','20-JAN-2026');
```

1 row created.

```

SQL> INSERT INTO REVIEW VALUES ('R00006','C006','P106',3,'Average product quality','22-JAN-2026');
1 row created.

SQL> INSERT INTO REVIEW VALUES ('R00007','C007','P107',5,'Fast and effective medicine','25-JAN-2026');
1 row created.

SQL> INSERT INTO REVIEW VALUES ('R00008','C009','P108',4,'Good medicine and packaging','27-JAN-2026');
1 row created.

SQL> INSERT INTO REVIEW VALUES ('R00009','C001','P109',5,'Excellent product','29-JAN-2026');
1 row created.

SQL> INSERT INTO REVIEW VALUES ('R00010','C002','P110',4,'Satisfied with the medicine','31-JAN-2026');
1 row created.

```

Displaying the Table after Insertion:

```

SQL> SELECT * FROM REVIEW;

```

REVIEWI	CUSTOME	PRODUCT	RATING	REVIEWTEXT	REVIEWDAT
R00001	C001	P101	5	Excellent medicine quality	10-JAN-26
R00002	C002	P102	4	Good product and effective	12-JAN-26
R00003	C003	P103	5	Very useful medicine	15-JAN-26
R00004	C004	P104	4	Good quality product	18-JAN-26
R00005	C005	P105	5	Highly satisfied with product	20-JAN-26

3. Store customer ratings from 1 to 5 stars.

```

SQL> ALTER TABLE REVIEW ADD CONSTRAINT CHK_Rating CHECK(Rating >= 1 AND Rating <= 5);
Table altered.

SQL> INSERT INTO REVIEW VALUES ('R00019','C001','P101',6,'Excellent product',SYSDATE);
INSERT INTO REVIEW VALUES ('R00019','C001','P101',6,'Excellent product',SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (C##ABINAYA.CHK_RATING) violated

```

4. Update review comments if required.

```
SQL> UPDATE REVIEW SET REVIEWTEXT='Good quality but could be improved' WHERE REVIEWID='R00003';
1 row updated.
```

5. Remove inappropriate or invalid reviews.

```
SQL> DELETE FROM REVIEW WHERE REVIEWID='R00010';

1 row deleted.
```

6. Retrieve Product Review Details:

Writing the SQL Queries for the following

1. Display all reviews for a product.

```
SQL> SELECT * FROM REVIEW WHERE PRODUCTID='P101';
```

REVIEWID	CUSTOMER	PRODUCT	RATING	REVIEWTEXT	REVIEWDATE
R00001	C001	P101	5	Excellent medicine quality	10-JAN-26

2. Display customer name with their reviews.

```
SQL> SELECT C.Customer_Name,R.ReviewID,R.ProductID,R.Rating,R.ReviewText FROM Customer C JOIN Review R ON C.Customer_ID=R.CustomerID;
```

CUSTOMER_NAME	REVIEWID	PRODUCT	RATING	REVIEWTEXT
Devi	R00001	P101	5	Excellent medicine quality
Radhi	R00002	P102	4	Good product and effective
Ajay	R00003	P103	5	Good quality but could be improved
Vijay	R00004	P104	4	Good quality product
Abi	R00005	P105	5	Highly satisfied with product
Anu	R00006	P106	3	Average product quality

CUSTOMER_NAME	REVIEWID	PRODUCT	RATING
REVIEWTEXT			
Sindhu	R00007	P107	5
Fast and effective medicine			
Sanjay	R00008	P108	4
Good medicine and packaging			
Devi	R00009	P109	5
Excellent product			
9 rows selected.			

3. Find products having maximum reviews.

```
SQL> SELECT P.PRODUCT_ID,P.PRODUCT_NAME,COUNT(R.REVIEWID) AS REVIEW_COUNT FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID,P.P
RODUCT_NAME ORDER BY REVIEW_COUNT DESC;
```

PRODUCT	PRODUCT_NAME	REVIEW_COUNT
P107	Antiseptic Cream	1
P101	Paracetamol	1
P104	Omeprazole	1
P109	Metformin	

```

-----
PRODUCT_NAME
-----
REVIEW_COUNT
-----
1

P106
Cough Syrup
1

P108

PRODUCT
-----
PRODUCT_NAME
-----
REVIEW_COUNT
-----
Bandage
1

P103
Cetirizine
1

PRODUCT
-----
PRODUCT_NAME
-----
REVIEW_COUNT
-----
P105
Vitamin C
1

P102
Amoxicillin
1

```

4. Display recent customer feedback.

```

SQL> SELECT * FROM (SELECT C.CUSTOMER_NAME,P.PRODUCT_NAME,R.RATING,R.REVIEWTEXT AS CUSTOMER_FEEDBACK,R.REVIEWDATE AS REVIEW_DATE FROM REVIEW R JOIN CUSTOMER
C ON R.CUSTOMERID=C.CUSTOMER_ID JOIN PRODUCT P ON R.PRODUCTID=P.PRODUCT_ID ORDER BY R.REVIEWDATE DESC) WHERE ROWNUM<=5;

```

```

CUSTOMER_NAME
-----
PRODUCT_NAME
-----
RATING CUSTOMER_FEEDBACK REVIEW_DA
-----
Devi
Metformin 5 Excellent product 29-JAN-26

Sanjay
Bandage 4 Good medicine and packaging 27-JAN-26

CUSTOMER_NAME
-----
PRODUCT_NAME
-----
RATING CUSTOMER_FEEDBACK REVIEW_DA
-----
Sindhu
Antiseptic Cream 5 Fast and effective medicine 25-JAN-26

Anu
Cough Syrup

```

```

CUSTOMER_NAME
-----
PRODUCT_NAME
-----
RATING CUSTOMER_FEEDBACK REVIEW_DA
-----
3 Average product quality 22-JAN-26

Abi
Vitamin C 5 Highly satisfied with product 20-JAN-26

```

5. Retrieve reviews with ratings above 4.

```
SQL> SELECT * FROM Review WHERE Rating > 4;
```

REVIEWID	CUSTOMER	PRODUCT	RATING	REVIEWTEXT	REVIEWDATE
R00001	C001	P101	5	Excellent medicine quality	10-JAN-26
R00003	C003	P103	5	Good quality but could be improved	15-JAN-26
R00005	C005	P105	5	Highly satisfied with product	20-JAN-26

REVIEWID	CUSTOMER	PRODUCT	RATING	REVIEWTEXT	REVIEWDATE
R00007	C007	P107	5	Fast and effective medicine	25-JAN-26
R00009	C001	P109	5	Excellent product	29-JAN-26

6. Calculate Average Product Ratings
Use aggregate functions to analyze ratings

TASKS:

1. Calculate average rating for each product.

```
SQL> SELECT P.PRODUCT_NAME,AVG(R.RATING) AS AVERAGE_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_NAME;
```

PRODUCT_NAME	AVERAGE_RATING
Amoxicillin	4
Metformin	5
Paracetamol	5

PRODUCT_NAME	AVERAGE_RATING
Vitamin C	5
Cough Syrup	3
Antiseptic Cream	5

2. Count the number of reviews per product.

```
SQL> SELECT P.PRODUCT_NAME,COUNT(R.RATING) AS COUNT_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_NAME;
```

```
PRODUCT_NAME
```

```
-----
```

```
COUNT_RATING
```

```
-----
```

```
Amoxicillin
```

```
1
```

```
Metformin
```

```
1
```

```
Paracetamol
```

```
1
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
COUNT_RATING
```

```
-----
```

```
Vitamin C
```

```
1
```

```
Cough Syrup
```

```
1
```

```
Antiseptic Cream
```

```
1
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
COUNT_RATING
```

```
-----
```

```
Bandage
```

```
1
```

3. Find highest-rated products.

```
SQL> SELECT P.PRODUCT_ID,P.PRODUCT_NAME,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING AVG(R.RATING)=(SELECT MAX(AVG_RATING) FROM (SELECT AVG(RATING) AS AVG_RATING FROM REVIEW GROUP BY PRODUCTID));
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
P107
```

```
Antiseptic Cream
```

```
5
```

```
-----
```

```
P101
```

```
Paracetamol
```

```
5
```

```
-----
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
P109
```

```
Metformin
```

```
5
```

```
-----
```

```
P103
```

```
Cetirizine
```

4. Identify products with average rating above 4.

```
SQL> SELECT P.PRODUCT_ID,P.PRODUCT_NAME,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING AVG(R.RATING)>3 ORDER BY AVERAGE_RATING DESC;
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
P107
```

```
Antiseptic Cream
```

```
5
```

```
-----
```

```
P101
```

```
Paracetamol
```

```
5
```

```
-----
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
P109
```

```
Metformin
```

```
5
```

```
-----
```

```
P103
```

```
Cetirizine
```

7. Identify Highly Rated Products

Generate product rating reports.

Report 1: Product Rating Analysis

1. Display:
2. Product name
3. Number of reviews
4. Average rating

```
SQL> SELECT P.PRODUCT_NAME, COUNT(R.REVIEWID) AS NUMBER_OF_REVIEWS, ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING FROM PRODUCT P LEFT JOIN REVIEW R ON P.PRODUCT_ID
=R.PRODUCTID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY AVERAGE_RATING DESC NULLS LAST;
```

PRODUCT_NAME	NUMBER_OF_REVIEWS	AVERAGE_RATING
Cetirizine	1	5
Paracetamol	1	5
Antiseptic Cream	1	5

PRODUCT_NAME	NUMBER_OF_REVIEWS	AVERAGE_RATING
Metformin	1	5
Vitamin C	1	5
Bandage	1	4

PRODUCT_NAME	NUMBER_OF_REVIEWS	AVERAGE_RATING
Omeprazole	1	4

Report 2: Customer Feedback Analysis

> Most reviewed products

```
SQL> SELECT P.PRODUCT_NAME, COUNT(R.REVIEWID) AS REVIEW_COUNT FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME O
RDER BY REVIEW_COUNT DESC;
```

PRODUCT_NAME	REVIEW_COUNT
Antiseptic Cream	1
Paracetamol	1
Omeprazole	1

PRODUCT_NAME	REVIEW_COUNT
Metformin	1
Cough Syrup	1
Bandage	1

PRODUCT_NAME	REVIEW_COUNT
Cetirizine	1
Vitamin C	1

Highly rated products

```
SQL> SELECT P.PRODUCT_NAME,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING AVG(R.RATING)>3 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_NAME	AVERAGE_RATING
Antiseptic Cream	5
Paracetamol	5
Metformin	5
Cetirizine	5
Vitamin C	5
Amoxicillin	4
Bandage	4
Omeprazole	4

Products requiring improvement

```
SQL> SELECT P.PRODUCT_NAME,COUNT(R.REVIEWID) AS REVIEW_COUNT,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM PRODUCT P JOIN REVIEW R ON P.PRODUCT_ID=R.PRODUCTID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING AVG(R.RATING)<=3 ORDER BY AVERAGE_RATING;
```

PRODUCT_NAME	REVIEW_COUNT	AVERAGE_RATING
Cough Syrup	1	3

Report 3: Rating Distribution

Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATING FROM REVIEW WHERE RATING = 5;

FIVE_STAR_RATING
-----
5
```

Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATING FROM REVIEW WHERE RATING = 4;

FOUR_STAR_RATING
-----
3
```

Number of low-rated products

```
SQL> SELECT COUNT(*) AS LOW_RATED_PRODUCTS FROM (SELECT PRODUCTID FROM REVIEW GROUP BY PRODUCTID HAVING AVG(RATING) < 3);

LOW_RATED_PRODUCTS
-----
0
```

Complete Rating

```
SQL> SELECT RATING,COUNT(*) AS NUMBER_OF_RATINGS FROM REVIEW GROUP BY RATING ORDER BY RATING DESC;

RATING  NUMBER_OF_RATINGS
-----
5
4
3
```

RATING	NUMBER_OF_RATINGS
5	5
4	3
3	1